

Rust 和 ArkTS 建控件谁快

English version

在 SUP-AL90 (HarmonyOS 6.1) 上实测，release 构建。想复现的话，在 app 里打开 **Performance** 页，或者直接看 `hilog`。方法都写在 `crates/splash-oh/src/bench.rs` 里。

结果

Rust 走 ArkUI NDK 比 ArkTS 快 2.6 到 3.2 倍，不是之前说的 45 倍。之前那个数是错的：只实测了 Rust 这边，ArkTS 那边是拿另一个项目的常数推算出来的。

A	Rust → ArkUI NDK	~17 μs/节点
B	ArkTS → typeNode	~52 μs/节点

两边建的是同一个原生节点。`typeNode.createNode(ctx, 'Text')` 和 NDK 的 `createNode(ARKUI_NODE_TEXT)` 最后都走到 libace 里同一个 `TextPattern`，属性也一样是四个。2000 个节点，每次跑 10 轮，一共测了 6 次。

这个"同一个节点"很关键：后面的布局、测量、绘制两边是同一份原生代码，可以约掉，所以只有"建"这一步有差别，这次测的也就是这一步。

差在哪

每次调用	Rust	ArkTS	倍数
<code>createNode</code>	14.0 – 17.3 μs	34.6 – 41.2 μs	2.1 – 2.6x
<code>setAttribute</code>	0.025 μs	1.15 μs	46x
一次 JS → native napi 调用	—	0.058 μs	—
一次空 JS 循环迭代	—	0.017 μs	—

桥本身只有 58 纳秒，不是瓶颈。ArkTS 设一个属性要 1.15 μs，其中过桥只占 5%，剩下 95% 都在 JS 这边：`attribute` 那个 `modifier` 对象、参数装箱、分发、校验。Rust 那 25 纳秒就是一次函数指针调用加填一个小结构体。

真正费时间的是 **JS 对象**。`typeNode.createNode` 除了建原生节点，还要建包装对象、注册 `finalizer`、维护跨语言引用，之后 GC 还得把这些收掉。波动也是这么来的：ArkTS 一轮一轮之间在 43 到 85 μs 之间跳，而 Rust 拆开单独测的那几个数，三位有效数字都不带变的。

下面这两行在几次运行之间几乎不动（属性是 1.146 / 1.149 / 1.152 / 1.154 / 1.158 / 1.162 μ s，过桥是 0.058 / 0.059 μ s），所以结论靠它们撑着，上面那些抖得厉害的整节点 数字不用背这个锅。

不是预热

按顺序排的十轮：

```
A Rust    13.4 12.9 15.5 17.9 16.0 15.9 20.9 24.6 20.5 16.7
B ArkTS   47.1 81.5 85.3 47.3 47.5 55.5 44.5 50.4 50.0 81.5
```

预热的话曲线应该一路往下。B 没往下，就是来回跳，看不出趋势。

但要是 不给事件循环留空隙、连着跑完，B 就变成一路往上：**37.9** $\rightarrow$ **87.4**。这就是堆里每轮攒 2000 个包装对象的样子。中间给事件循环留出空隙，曲线就从"爬坡"变成"抖动"。是 GC，不是 JIT。

（A 也在慢慢往上飘，但它这条路上根本没有分配器，那是设备在发热。两边是在同一个 tick 里交替跑的，就是为了让这种影响平摊到双方头上。）

那为什么还要用 NDK

其实不是图快。是因为 napi 的延迟跟 JS 线程忙不忙直接相关，而建控件树本身就会把 JS 线程占满。

octos-one 那边测出来一次往返 1.05 ms，其中 730 μ s 纯粹是在排队等。同样一次跨越，线程空着的时候只要 31 μ s 左右。两个数都没测错，是我把线程争用当成桥的开销了。

所以：建控件快 3 倍是一方面，更要紧的是它不排队 —— UI 线程越忙，这个差距只会越大。这个说法比这个仓库一开始的说法弱得多，但它是数据能支撑的那个。

局限

- **测量 C (napi 往返) 在现在这版跑不起来。** 把整套测试改成由 ArkTS 一个 tick 跑一轮之后，JS 那边就不再确认 worker 发过去的消息了，原因还没查出来。README 里标了那几个数是旧版测的。worker 现在会超时报错，不会像以前那样一直挂着。
- **28 个页面只看了 6 个**，剩下 22 个没逐个验证。用的是同一套 DSL 辅助函数和同一个遍历器，但确实没看。
- **没在负载下测过。** 而"线程争用"这个说法到底成不成立，恰恰得在负载下才能验证，空闲状态的 microbenchmark 说明不了问题。

- ArkTS 的声明式那套完全没测。 `typeNode` 是它的命令式口子，选它就是因为只有它能跟 NDK 做等价对比。

三个测量上的坑

每一个都给出了看着挺像回事、其实是错的数字：

1. **拿常数推算。** ArkTS 那边压根没测过，是 `1051  $\mu$ s × 控件数` 算出来的——那个 1051 还是从另一个 app 里测的往返时间，而且默认了“每个控件过一次桥”这个从没验证过的前提。
2. **相减的结果比噪声还小。** 用（完整试验 – 只建节点）去算每个属性的开销，Rust 那边算出来是负数。信号本身不到 `1  $\mu$ s`，而两次 2000 节点试验之差的噪声底是它的好几倍。现在属性是单独直接测的。
3. **事件循环被卡住。** 在 `mount()` 里跑 benchmark，把 JS 线程占得太久，定时器队列直接不转了，结果整个 ArkTS 那半套测试静默地根本没跑——不报错、不输出，就是没结果。现在每一轮都由 ArkTS 驱动，一个 tick 跑一轮。

还有个小的：ArkTS 的 `@Component struct` 里声明的 `static` 成员，运行时读出来是 `undefined`，于是状态机一路穿下去也不报错。改成模块级 `const` 就好了。